Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Lehrstuhl für Informatik VIII
Informationstechnik für Luft- und Raumfahrt
Prof. Dr.-Ing. Sergio Montenegro

Introduction to RODOS

Online-Floatsat

Winter Term 2020/2021

Frank Flederer

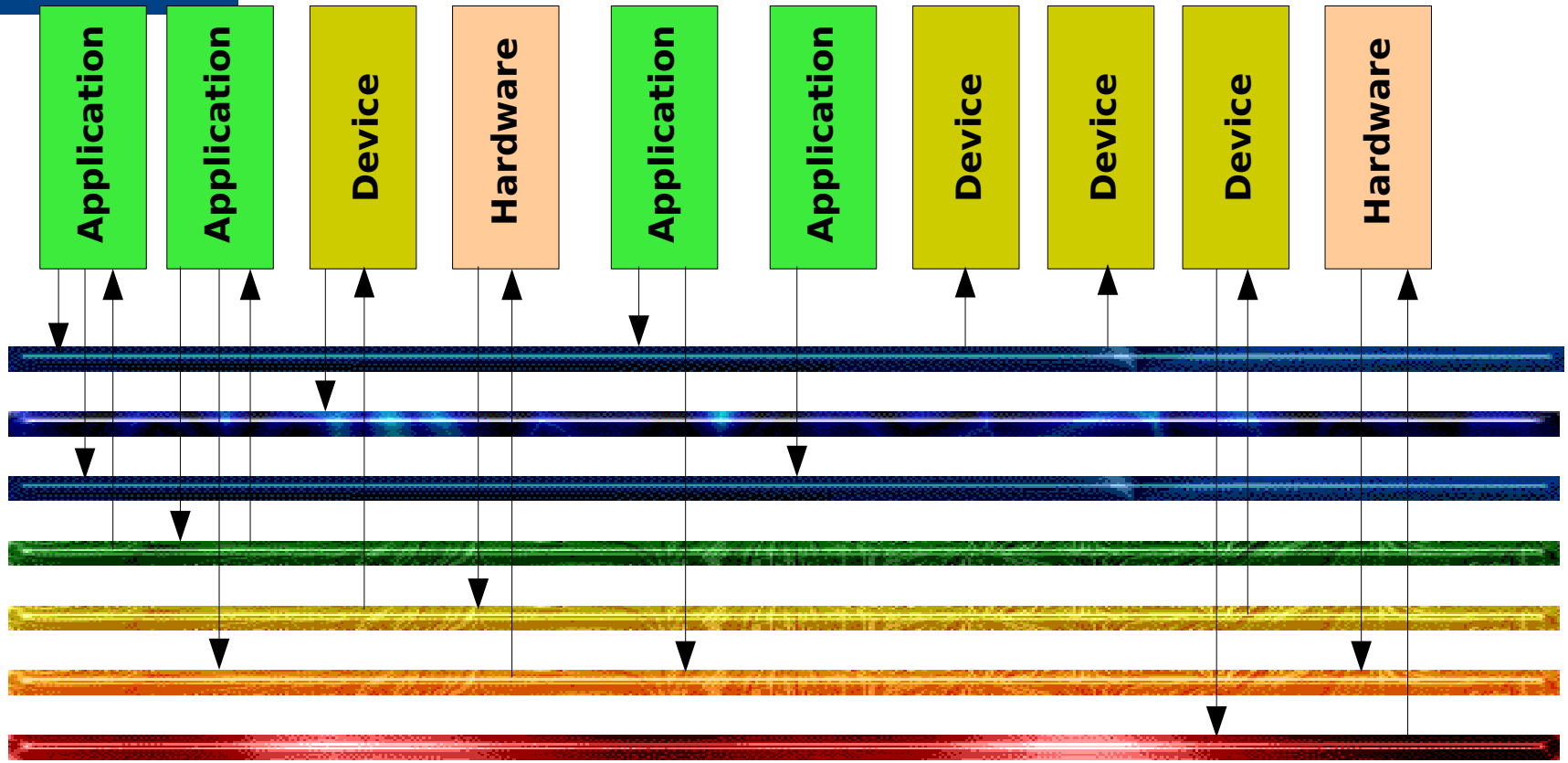
E-Mail: frank.flederer@uni-wuerzburg.de

Tel.: +49 931 31-85120

Publish / Subscribe

- Topics represent communication channels with an ID and a given data type
- Subscribers register for topics
- Publishers send messages to topics
- Published messages are forwarded to all subscribers

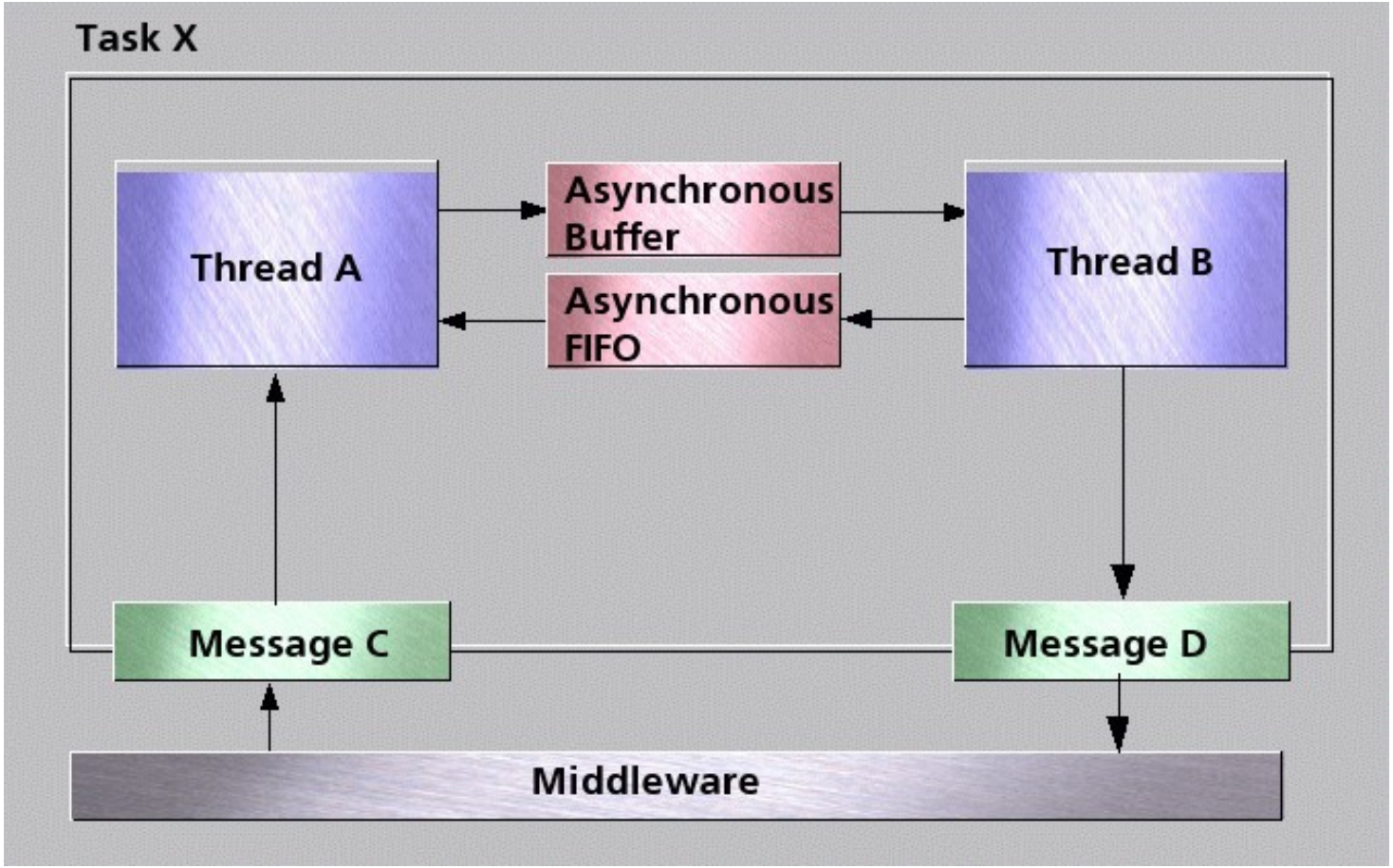
Publish / Subscribe



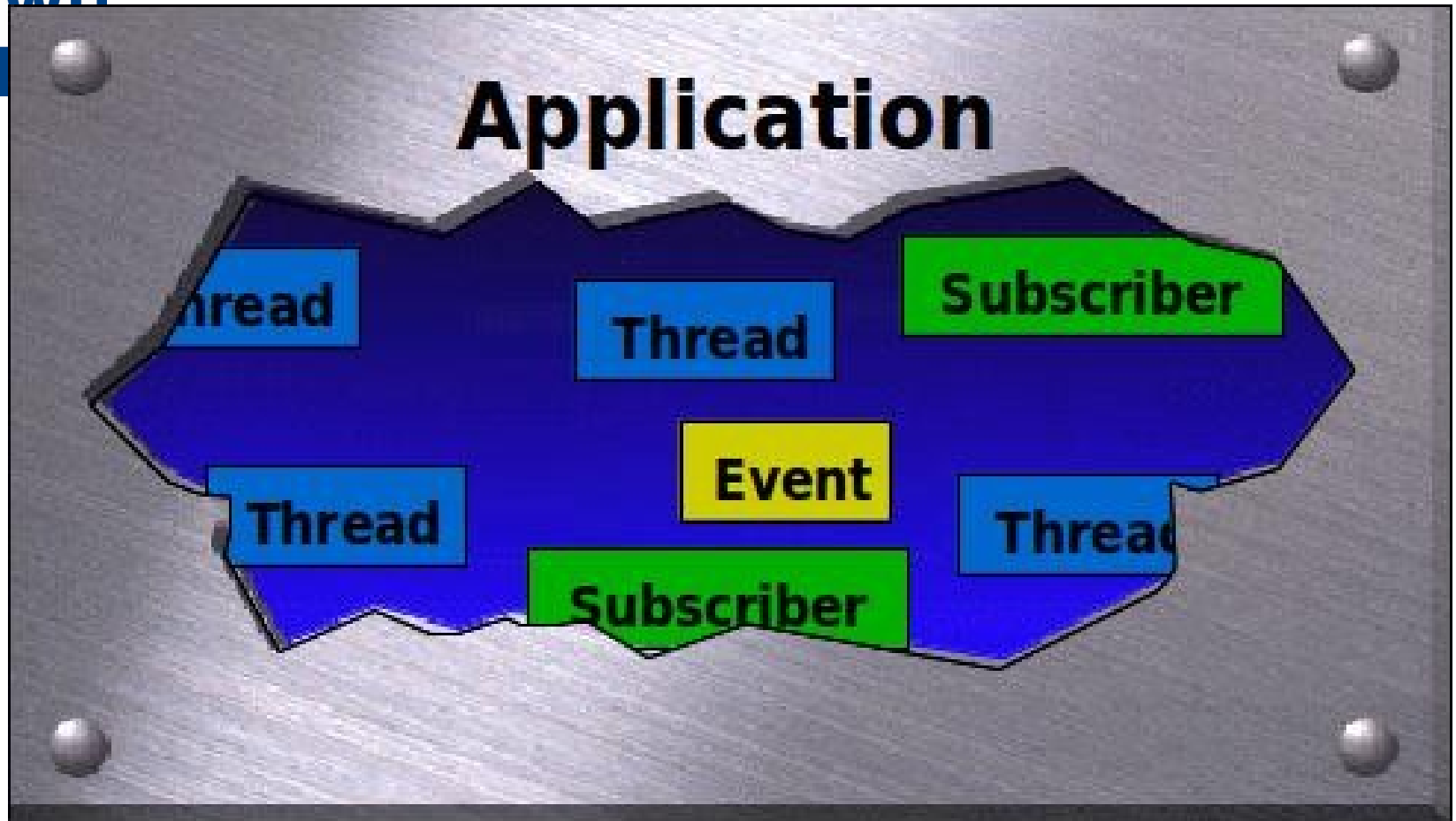
Topics ~=

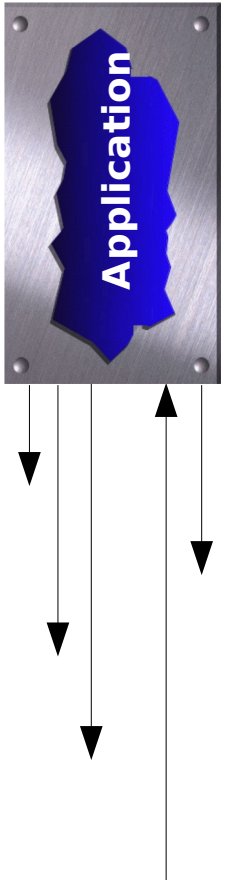
Communication Channels

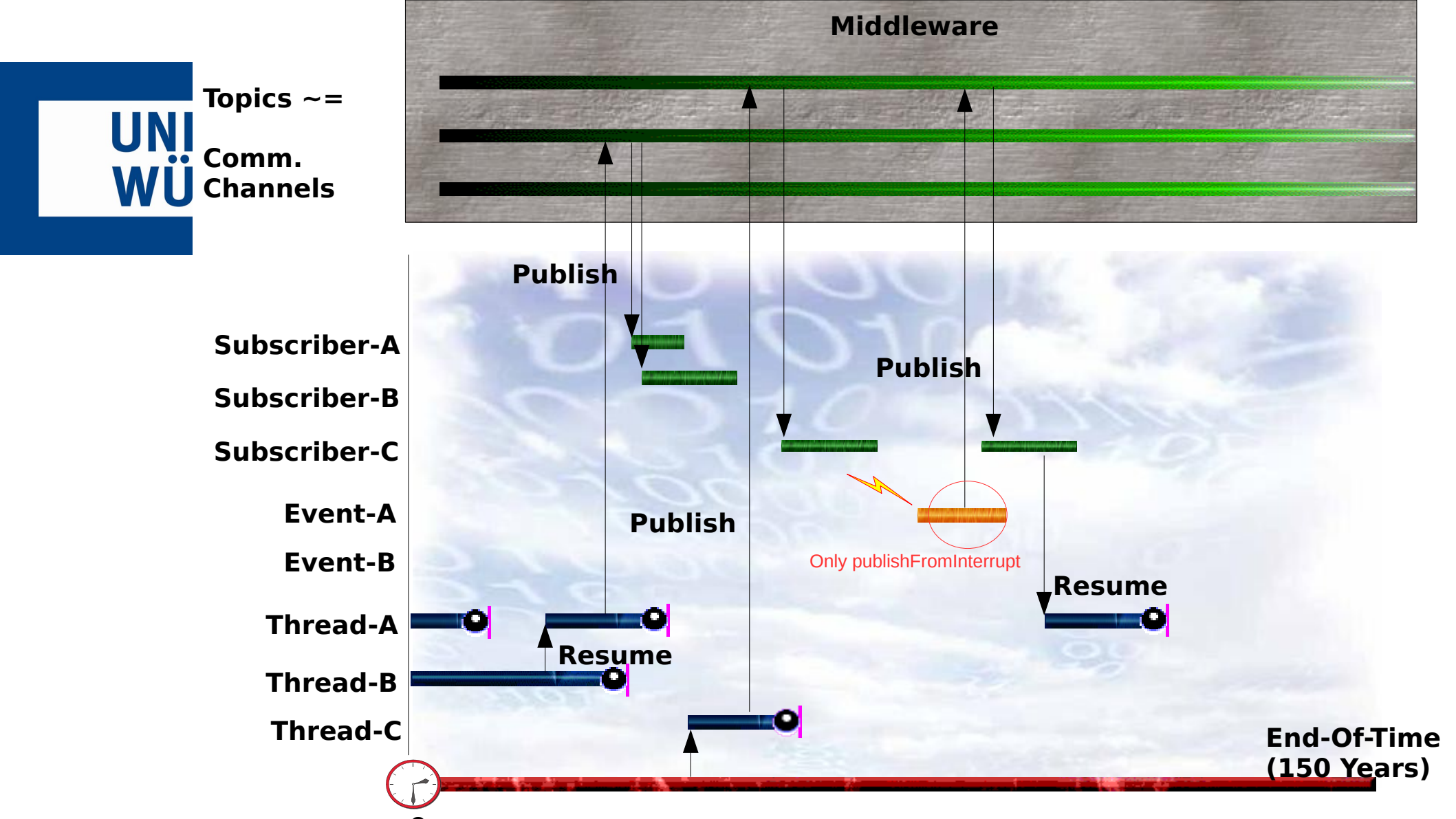
Communication between Tasks



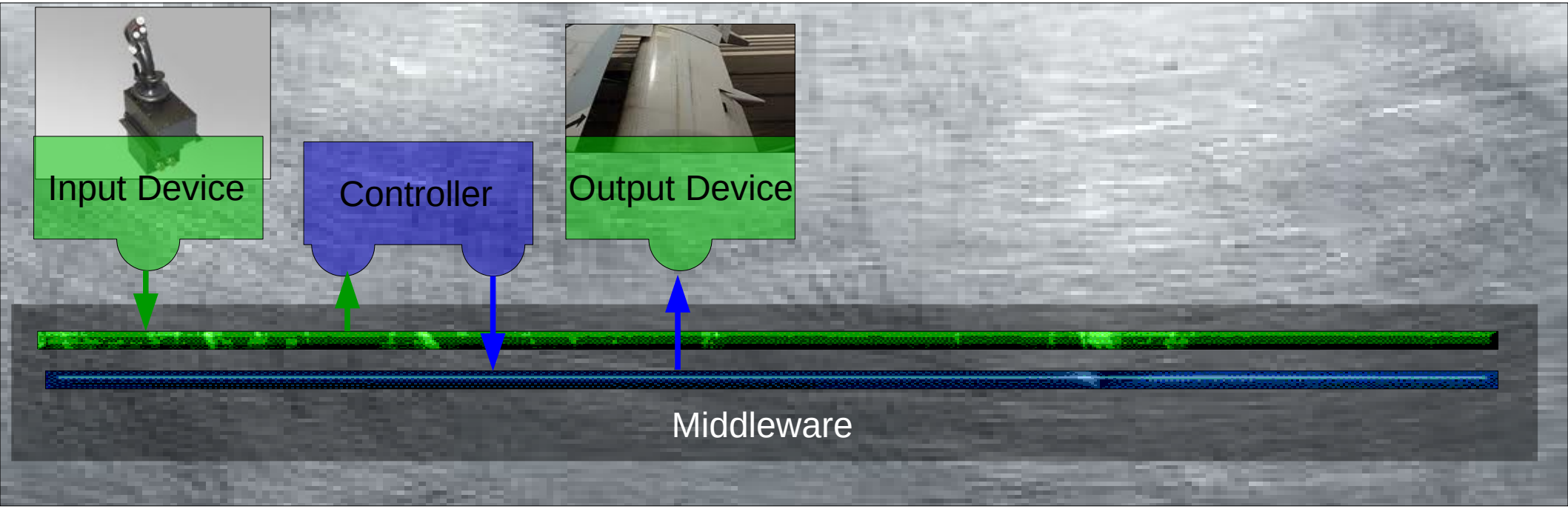
Applications



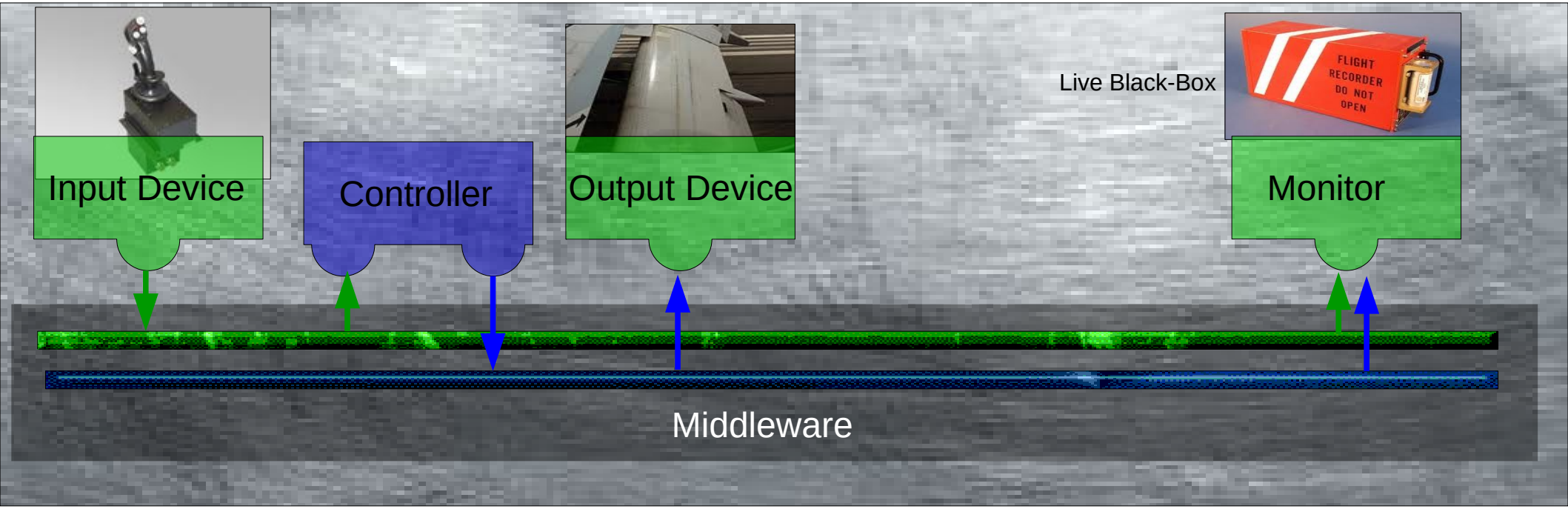



Middleware
Topics ~=
Comm.
Channels
Publish
Publish
Publish
Subscriber-A
Subscriber-B
Subscriber-C
Event-A
Event-B
Only publishFromInterrupt
Resume
Thread-A
Resume
Thread-B
Thread-C
End-Of-Time
(150 Years)

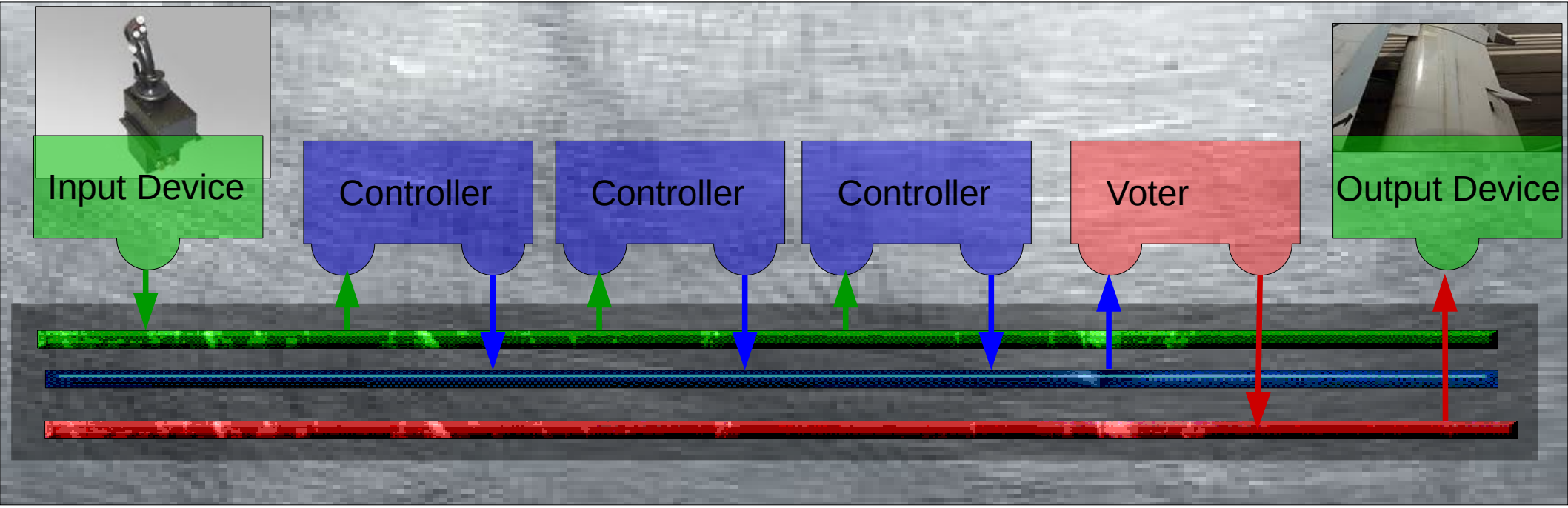
Dynamic Composition



Dynamic Composition



Dynamic Composition



Topic

```
Topic<Type>::Topic(int64_t id, const char* name, bool _onlyLocal = false)
```

- ID is given in the constructor
- Type is defined as a template parameter
- Name as string (for debugging)
- Flag onlyLocal for not forwarding it via gateway (see later)

Publisher

- No special registering necessary
- Just invoke Topic::publish method

```
publish(Type &msg, bool shallSendToNetwork = true)
```

  - msg: value to send
  - shallSendToNetwork: if the message should be forwarded via gateway (see later)

```
void publishFromInterrupt(void *any, size_t len = 0);
```

  - no thread safety

Subscriber

- Different ways to implement subscribers
- Main class: RODOS::Subscriber

1a) Subscriber Subclass

```
RODOS::Topic<int> myTopic(123, "MyTopic");

class MySubscriber : RODOS::Subscriber {
public:
  MySubscriber() : RODOS::Subscriber(myTopic, "MySubscriber") {}

  uint32_t put(const uint32_t topicId, const size_t len, void* data,
               const NetMsgInfo& netMsgInfo) override {
    int* intData = reinterpret_cast<int*>(data);
    // process intData
  }
};

MySubscriber mySubscriber;
```

1b) Subscriber Subclass

```
RODOS::Topic<int> myTopic(123, "MyTopic");

class MySubscriber : RODOS::SubscriberReceiver<int> {
public:
  MySubscriber() : RODOS::SubscriberReceiver(myTopic, "MySubscriber") {}

  uint32_t put(int& data) override {
    int* intData = reinterpret_cast<int*>(data);
    // process intData
  }
};

MySubscriber mySubscriber;
```

2a) Passing Putter

```
RODOS::Topic<int> myTopic(123, "MyTopic");

class MyPutter : RODOS::Putter {
public:
  uint32_t put(const uint32_t topicId, const size_t len, void* data,
               const NetMsgInfo& netMsgInfo) override {
    int* intData = reinterpret_cast<int*>(data);
    // process intData
  }
};

MyPutter myPutter;

Rodos::Subscriber mySubscriber(myTopic, myPutter, "MySubscriber");
```

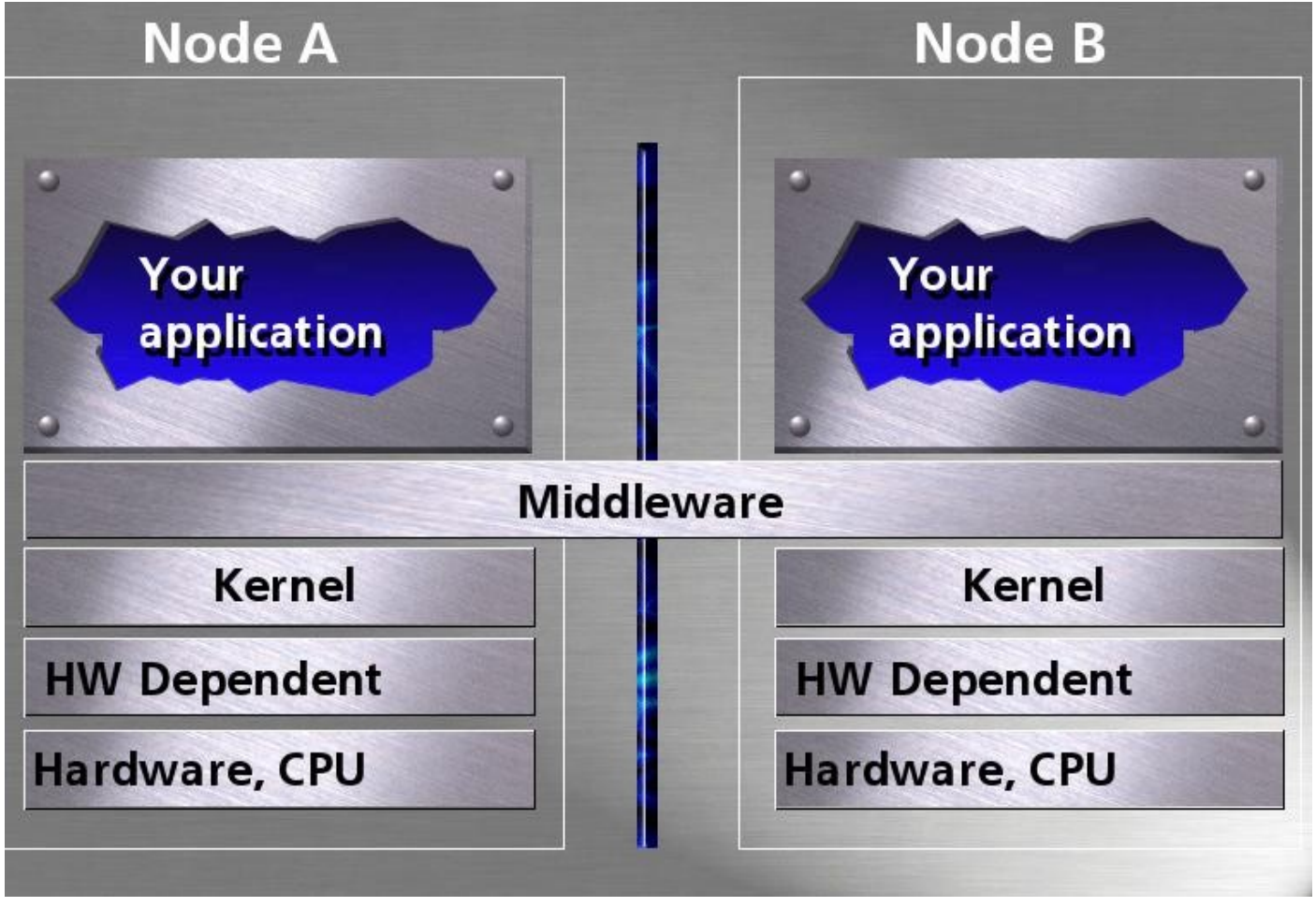
2b) Passing Putter

```
RODOS::Topic<int> myTopic(123, "MyTopic");

RODOS::SyncFifo<int, 16> myFifo;

Rodos::Subscriber myFifoSubscriber(myTopic, myFifo, "MyFifoSubscriber");

RODOS::CommBuffer<int> myBuffer;

Rodos::Subscriber myBufferSubscriber(myTopic, myBuffer, "MyFifoSubscriber");
```

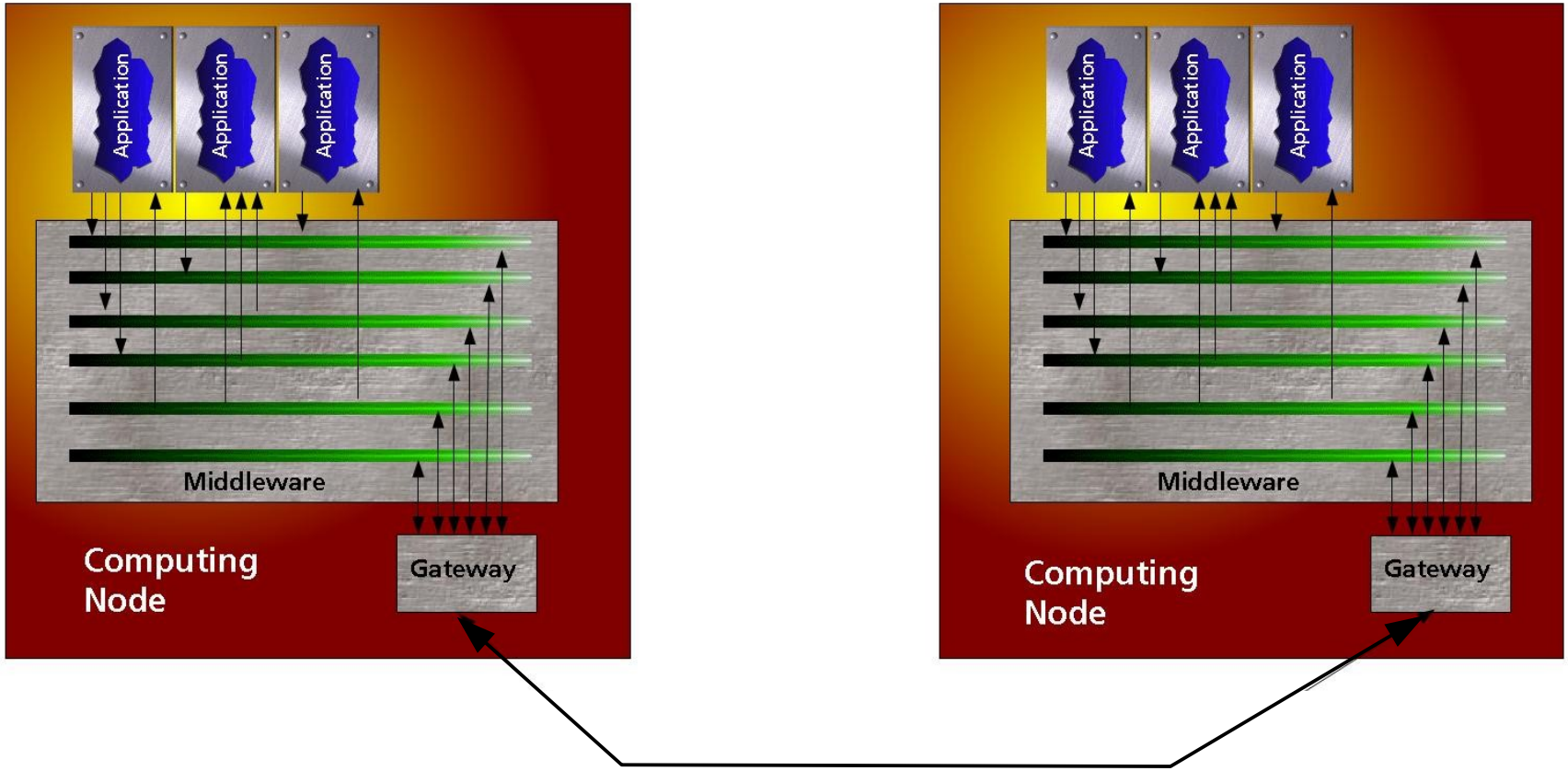
Gateway

- Communication between different computers
- Rodos supports many hardware interfaces and busses
- Own software protocol

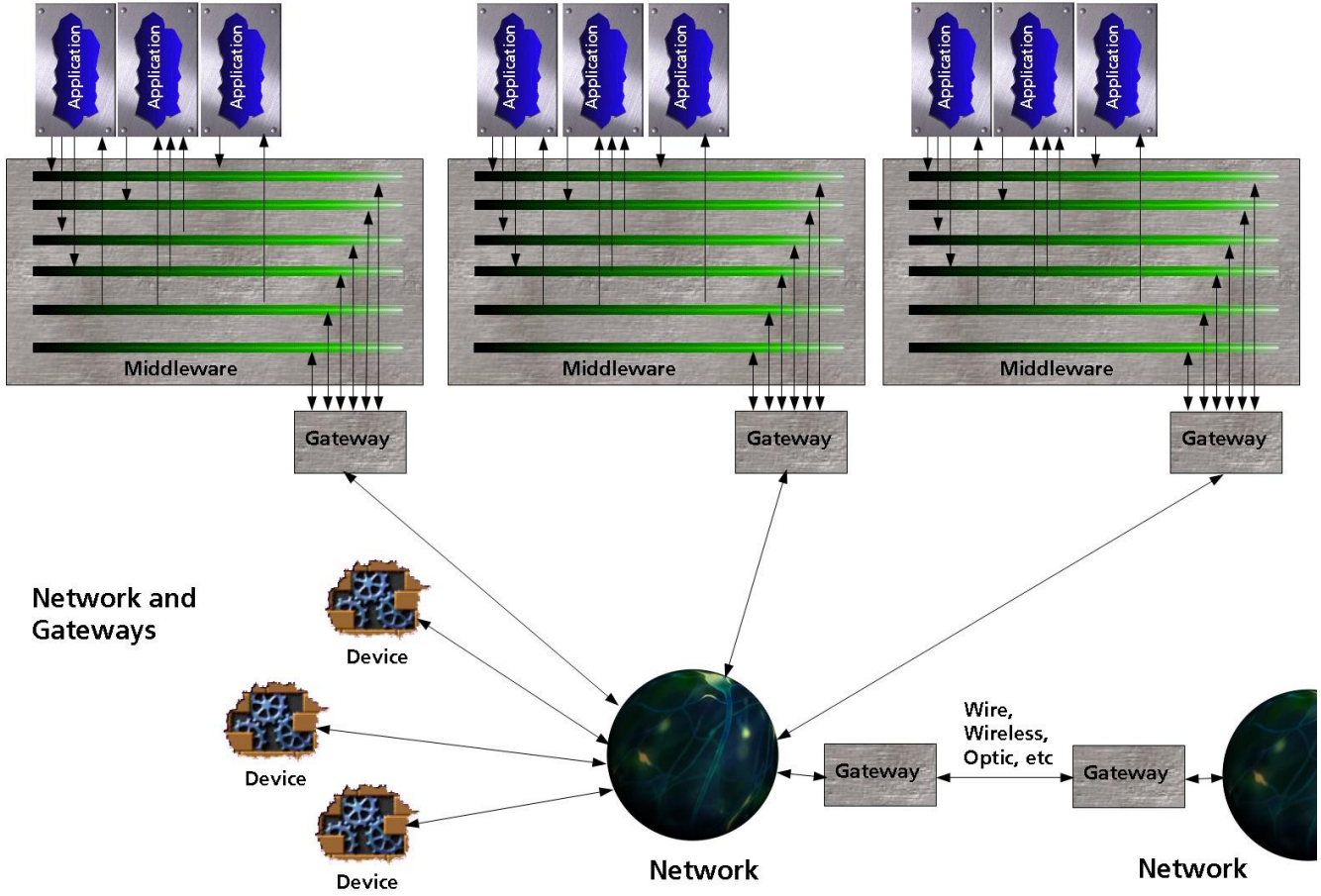
Gateway


Node A
Node B
Your application
Your application
Middleware
Kernel
Kernel
HW Dependent
HW Dependent
Hardware, CPU
Hardware, CPU

Gateway


Application
Application
Application
Middleware
Computing Node
Gateway
Application
Application
Application
Middleware
Computing Node
Gateway

Gateway


Application
Application
Application
Middleware
Gateway
Application
Application
Application
Middleware
Gateway
Application
Application
Application
Middleware
Gateway
Network and
Gateways
Device
Device
Device
Network
Gateway
Wire,
Wireless,
Optic, etc
Gateway
Network

Gateway

- Three classes are required for instantiating a gateway
  - HAL instance for the bus
  - Linkinterface instance for the translation between Rodos middleware and the bus protocol
  - Gateway instance connecting Rodos Middleware and the Linkinterface

Gateway Instantiation

```
RODOS::HAL_UART uart(RODOS::UART_IDX1);
RODOS::LinkinterfaceUART linkinterfaceUart(&uart);
RODOS::Gateway gatewayUart(&linkinterfaceUart);
```

```
RODOS::UDPInOut udp(1337);
RODOS::LinkinterfaceUART linkinterfaceUdp(&udp);
RODOS::Gateway gatewayUdp(&linkinterfaceUart);
```

Gateway Configuration

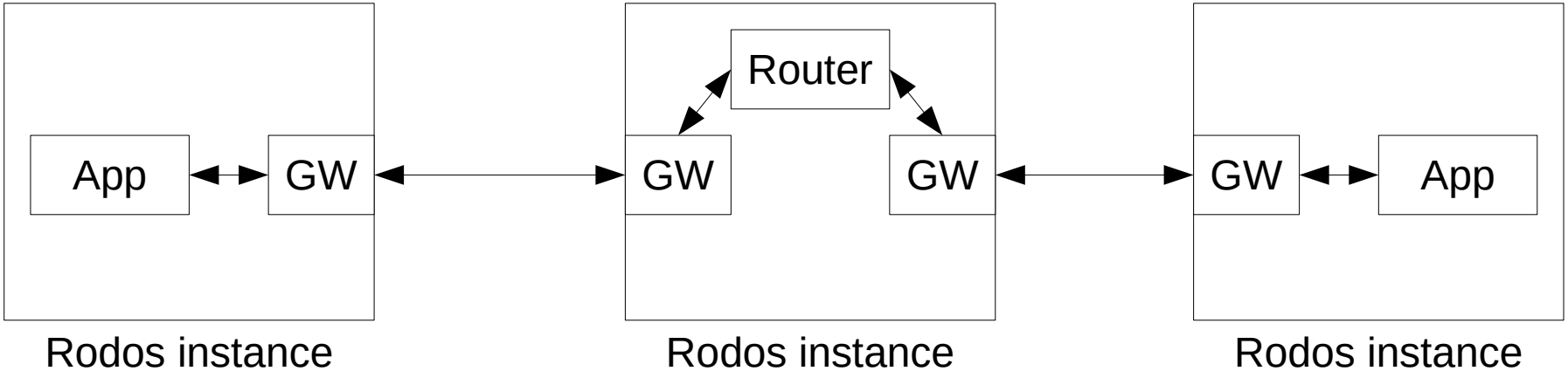
```
extern RODOS::Topic<int> myFirstTopic;
extern RODOS::Topic<double> mySecondTopic;
extern RODOS::Topic<bool> myThirdTopic;
extern RODOS::Gateway gateway;

gateway.addTopicsToForward(
  myFirstTopic, myThirdTopic);


gateway.setForwardAllTopics(true);
```

Select which topics to forward...

… or just forward all topics

Router

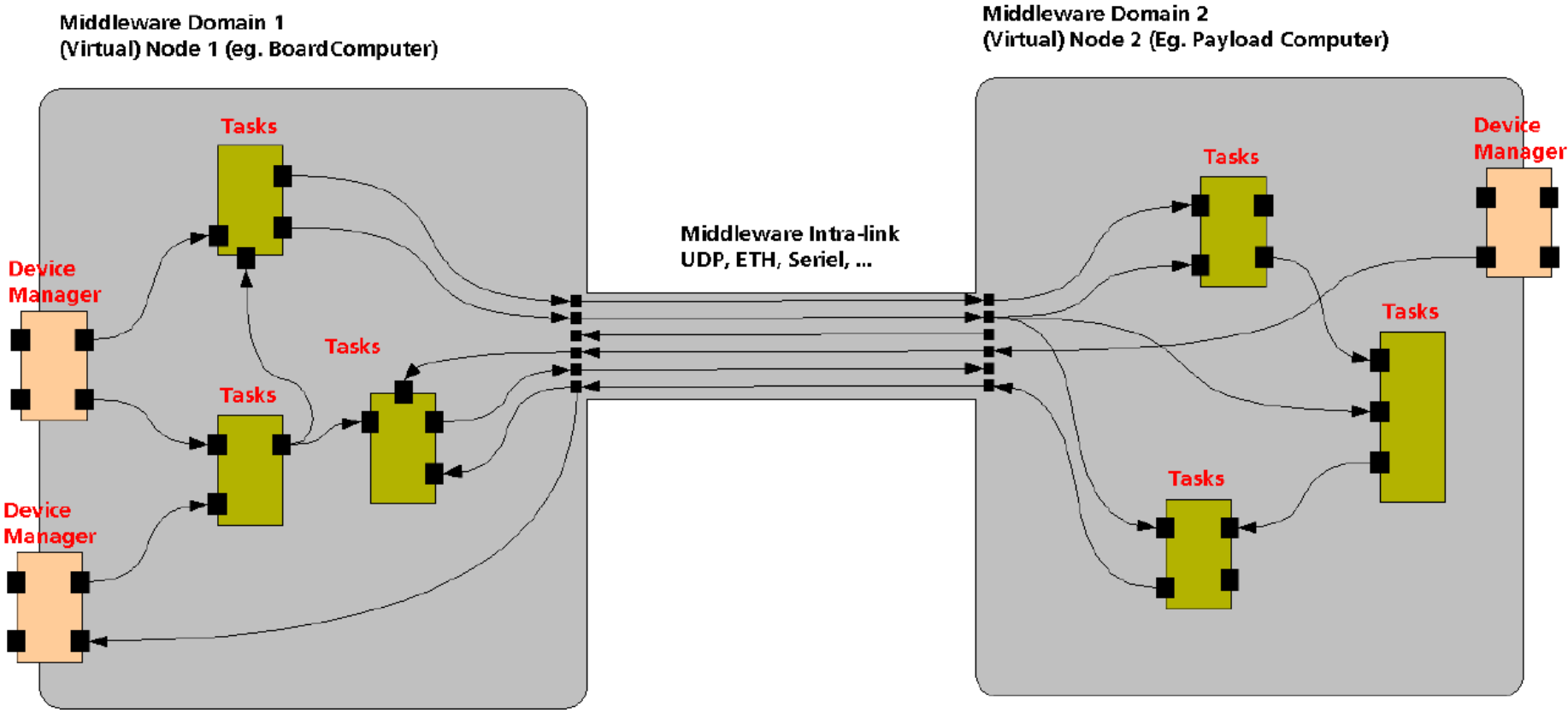
- Gateways just translate between local topics and a communication interface
- To forward between gateways: use Router

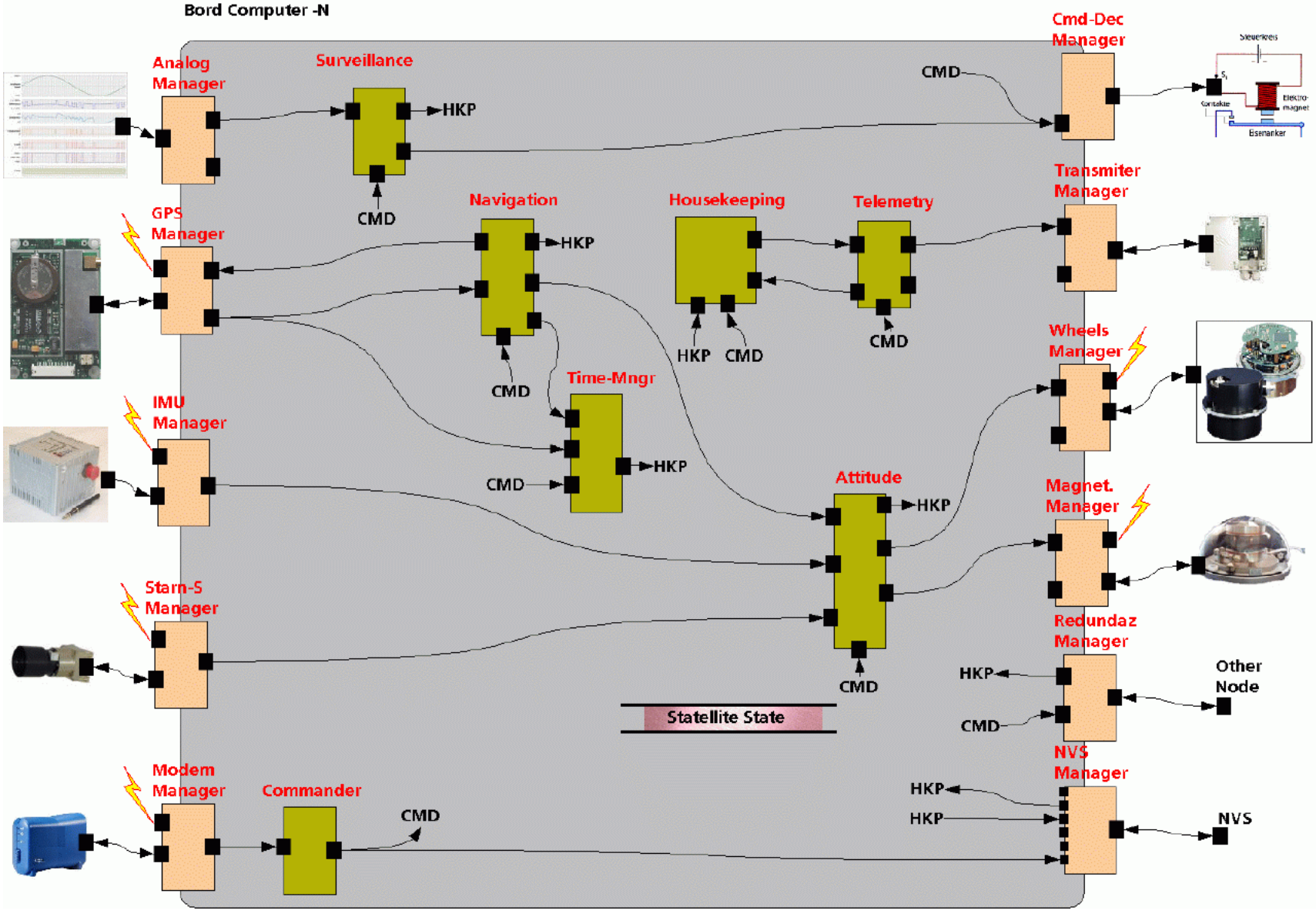


Router

```
extern RODOS::Gateway myFirstGateway;
extern RODOS::Gateway mySecondGateway;

RODOS::Router router;
router.addGateway(&myFirstGateway);
router.addGateway(&mySecondGateway);
```

Middleware Example




Bord Computer -N
Analog Manager
Surveillance
HKP
CMD
Cmd-Dec Manager
CMD
GPS Manager
Navigation
HKP
CMD
Housekeeping
HKP
CMD
Telemetry
CMD
Transmiter Manager
Time-Mngr
HKP
CMD
IMU Manager
Wheels Manager
Attitude
HKP
CMD
Magnet. Manager
Starn-S Manager
Statellite State
Redundaz Manager
HKP
CMD
Other Node
Modem Manager
Commander
CMD
NVS Manager
HKP
HKP
NVS

Thank you for your attention!

Q&A sessions on

- Tuesday 15:00 UTC
- Thursday 13:00 UTC

Frank Flederer

E-Mail: frank.flederer@uni-wuerzburg.de

Tel.: +49 931 31-85120